

```

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import pandas as pd
import time
from sklearn.decomposition import PCA
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import accuracy_score, precision_score, f1_score, ConfusionMatrixDisplay, roc_curve, auc
from sklearn.preprocessing import label_binarize
from sklearn.utils.class_weight import compute_sample_weight
from models import ResidualUnit, prep_for_cnn, preprocess_for_cnn

data = np.load('octmnist.npz')
train_images = data["train_images"]
train_labels = data["train_labels"].ravel()
test_images = data["test_images"]
test_labels = data["test_labels"].ravel()

class_names = ["Normal", "CNV", "DME", "Drusen"]
sample_weights_full = compute_sample_weight(class_weight="balanced", y=train_labels)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)

print(f"Train: {train_images.shape} Test: {test_images.shape}")

```

Train: (97477, 28, 28) Test: (1000, 28, 28)

```

from sklearn.model_selection import StratifiedShuffleSplit

def make_datasets(x, y, sample_weights=None, batch_size=128, val_split=0.1, seed=42):
    tr_idx, va_idx = next(StratifiedShuffleSplit(1, test_size=val_split, random_state=seed).split(x, y))

    x_tr, y_tr = x[tr_idx], y[tr_idx]
    x_va, y_va = x[va_idx], y[va_idx]

    if sample_weights is not None:
        train_ds = tf.data.Dataset.from_tensor_slices((x_tr, y_tr, sample_weights[tr_idx]))
    else:
        train_ds = tf.data.Dataset.from_tensor_slices((x_tr, y_tr))

    train_ds = train_ds.shuffle(len(x_tr), seed=seed).batch(batch_size)
    val_ds = tf.data.Dataset.from_tensor_slices((x_va, y_va)).batch(batch_size)

    return train_ds, val_ds

```

This helper function was necessary as I was getting a version compatibility issue when using validation_split=.1

```

def evaluate_classifier(name, y_true, y_pred, y_proba, elapsed):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average="weighted", zero_division=1)
    f1 = f1_score(y_true, y_pred, average="weighted", zero_division=1)

    print(f"{name}")
    print(f"Accuracy: {acc:.4f}")
    print(f"Weighted Precision: {prec:.4f}")
    print(f"Weighted F1: {f1:.4f}")
    print(f"Training Time: {elapsed:.1f}s")

    fig, ax = plt.subplots(1, 2, figsize=(14, 5))

    ConfusionMatrixDisplay.from_predictions(
        y_true, y_pred,
        display_labels=class_names,
        ax=ax[0], colorbar=False, cmap="plasma",
    )
    ax[0].set_title(f"{name} - Confusion Matrix")
    ax[0].tick_params(axis="x", rotation=15)

    y_bin = label_binarize(y_true, classes=[0, 1, 2, 3])
    colors = ["r", "b", "g", "y"]
    aucs = []
    for i, (cname, col) in enumerate(zip(class_names, colors)):
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
        roc_auc = auc(fpr, tpr)
        aucs.append(roc_auc)
        ax[1].plot(fpr, tpr, color=col, lw=1.8, label=f"{cname} (AUC={roc_auc:.3f})")
    ax[1].plot([0, 1], [0, 1], "k--", lw=1)
    ax[1].set_xlabel("False Positive Rate")
    ax[1].set_ylabel("True Positive Rate")

```

```

ax[1].set_title(f"{name} - ROC Curves (OvR)")
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()

mean_auc = np.mean(aucs)
return {"name": name, "acc": acc, "prec": prec, "f1": f1, "time": elapsed, "auc": mean_auc}

```

```

x_tr_flat = train_images.reshape(len(train_images), -1).astype(np.float32)
x_te_flat = test_images.reshape(len(test_images), -1).astype(np.float32)

scaler_wd = MinMaxScaler()
x_tr_s = scaler_wd.fit_transform(x_tr_flat)
x_te_s = scaler_wd.transform(x_te_flat)

pca_90 = PCA(n_components=0.90, random_state=42).fit(x_tr_s)
n_wide = pca_90.n_components_
print(f"PCA(0.90) → {n_wide} components")

x_tr_pca = pca_90.transform(x_tr_s).astype(np.float32)
x_te_pca = pca_90.transform(x_te_s).astype(np.float32)

scaler_px = StandardScaler()
x_tr_flat_norm = scaler_px.fit_transform(x_tr_flat)
x_te_flat_norm = scaler_px.transform(x_te_flat)

x_tr_wd = np.concatenate([x_tr_pca, x_tr_flat_norm], axis=1).astype(np.float32)
x_te_wd = np.concatenate([x_te_pca, x_te_flat_norm], axis=1).astype(np.float32)
print(f"Wide & Deep input shape: {x_tr_wd.shape}")

```

```

PCA(0.90) → 21 components
Wide & Deep input shape: (97477, 805)

```

```

def build_best_wide_deep(n_wide, n_train_samples, batch_size=128, epochs=100):
    learning_rate = 0.002
    deep_units = 256
    n_deep_layers = 2
    activation = "relu"
    dropout_rate = 0.1

    steps_per_epoch = n_train_samples // batch_size
    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=0.0,
        decay_steps=steps_per_epoch * epochs,
        warmup_target=learning_rate,
        warmup_steps=steps_per_epoch * 2,
    )

    inputs = tf.keras.layers.Input(shape=(n_wide + 784,))
    wide = inputs[:, :n_wide]
    deep = inputs[:, n_wide:]

    for _ in range(n_deep_layers):
        deep = tf.keras.layers.Dense(deep_units, activation=activation)(deep)
        deep = tf.keras.layers.Dropout(dropout_rate)(deep)

    concat = tf.keras.layers.Concatenate()([wide, deep])
    outputs = tf.keras.layers.Dense(4, activation="softmax")(concat)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

wd_model = build_best_wide_deep(n_wide, n_train_samples=len(train_images))
wd_model.summary()

```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 805)	0	-
get_item_3 (GetItem)	(None, 784)	0	input_layer_1[0]...
dense_3 (Dense)	(None, 256)	200,960	get_item_3[0][0]
dropout_2 (Dropout)	(None, 256)	0	dense_3[0][0]
dense_4 (Dense)	(None, 256)	65,792	dropout_2[0][0]
get_item_2 (GetItem)	(None, 21)	0	input_layer_1[0]...
dropout_3 (Dropout)	(None, 256)	0	dense_4[0][0]
concatenate_1 (Concatenate)	(None, 277)	0	get_item_2[0][0], dropout_3[0][0]
dense_5 (Dense)	(None, 4)	1,112	concatenate_1[0]...

Total params: 267,864 (1.02 MB)

```
train_ds_wd, val_ds_wd = make_datasets(
    x_tr_wd, train_labels, sample_weights=sample_weights_full, batch_size=128
)
```

```
start = time.time()
wd_model.fit(train_ds_wd, validation_data=val_ds_wd, epochs=100, callbacks=[early_stopping_cb], verbose=1)
t = time.time() - start
```

```
wd_proba = wd_model.predict(x_te_wd)
wd_pred = np.argmax(wd_proba, axis=1)
```

```
Epoch 1/100
686/686 ————— 12s 8ms/step - accuracy: 0.4822 - loss: 1.1956 - val_accuracy: 0.5927 - val_loss: 0.9490
Epoch 2/100
686/686 ————— 3s 4ms/step - accuracy: 0.6127 - loss: 0.9694 - val_accuracy: 0.6737 - val_loss: 0.8020
Epoch 3/100
686/686 ————— 2s 3ms/step - accuracy: 0.6427 - loss: 0.9000 - val_accuracy: 0.6594 - val_loss: 0.8051
Epoch 4/100
686/686 ————— 2s 3ms/step - accuracy: 0.6608 - loss: 0.8460 - val_accuracy: 0.6717 - val_loss: 0.7825
Epoch 5/100
686/686 ————— 2s 3ms/step - accuracy: 0.6751 - loss: 0.8085 - val_accuracy: 0.6992 - val_loss: 0.7113
Epoch 6/100
686/686 ————— 2s 3ms/step - accuracy: 0.6814 - loss: 0.7860 - val_accuracy: 0.7179 - val_loss: 0.7049
Epoch 7/100
686/686 ————— 3s 4ms/step - accuracy: 0.6887 - loss: 0.7607 - val_accuracy: 0.6372 - val_loss: 0.7751
Epoch 8/100
686/686 ————— 2s 4ms/step - accuracy: 0.6897 - loss: 0.7442 - val_accuracy: 0.6612 - val_loss: 0.7770
Epoch 9/100
686/686 ————— 2s 3ms/step - accuracy: 0.6998 - loss: 0.7209 - val_accuracy: 0.7121 - val_loss: 0.7004
Epoch 10/100
686/686 ————— 2s 3ms/step - accuracy: 0.7063 - loss: 0.7052 - val_accuracy: 0.7035 - val_loss: 0.6932
Epoch 11/100
686/686 ————— 2s 3ms/step - accuracy: 0.7136 - loss: 0.6849 - val_accuracy: 0.7236 - val_loss: 0.6706
Epoch 12/100
686/686 ————— 3s 4ms/step - accuracy: 0.7181 - loss: 0.6763 - val_accuracy: 0.7127 - val_loss: 0.6808
Epoch 13/100
686/686 ————— 3s 4ms/step - accuracy: 0.7208 - loss: 0.6638 - val_accuracy: 0.7271 - val_loss: 0.6679
Epoch 14/100
686/686 ————— 2s 3ms/step - accuracy: 0.7262 - loss: 0.6443 - val_accuracy: 0.7597 - val_loss: 0.5922
Epoch 15/100
686/686 ————— 2s 3ms/step - accuracy: 0.7346 - loss: 0.6350 - val_accuracy: 0.7242 - val_loss: 0.6497
Epoch 16/100
686/686 ————— 2s 3ms/step - accuracy: 0.7381 - loss: 0.6146 - val_accuracy: 0.7061 - val_loss: 0.6696
Epoch 17/100
686/686 ————— 3s 3ms/step - accuracy: 0.7394 - loss: 0.6070 - val_accuracy: 0.7388 - val_loss: 0.6243
Epoch 18/100
686/686 ————— 3s 4ms/step - accuracy: 0.7453 - loss: 0.5972 - val_accuracy: 0.7147 - val_loss: 0.6516
Epoch 19/100
686/686 ————— 3s 4ms/step - accuracy: 0.7470 - loss: 0.5892 - val_accuracy: 0.7499 - val_loss: 0.6119
Epoch 20/100
686/686 ————— 2s 3ms/step - accuracy: 0.7519 - loss: 0.5768 - val_accuracy: 0.7232 - val_loss: 0.6799
Epoch 21/100
686/686 ————— 2s 3ms/step - accuracy: 0.7547 - loss: 0.5713 - val_accuracy: 0.7334 - val_loss: 0.6442
Epoch 22/100
686/686 ————— 2s 3ms/step - accuracy: 0.7582 - loss: 0.5581 - val_accuracy: 0.7248 - val_loss: 0.6550
Epoch 23/100
686/686 ————— 3s 4ms/step - accuracy: 0.7642 - loss: 0.5455 - val_accuracy: 0.7135 - val_loss: 0.6642
Epoch 24/100
686/686 ————— 2s 3ms/step - accuracy: 0.7655 - loss: 0.5425 - val_accuracy: 0.7536 - val_loss: 0.6155
32/32 ————— 1s 13ms/step
```

```
x_tr_cnn = preprocess_for_cnn(train_images)
x_te_cnn = preprocess_for_cnn(test_images)
```

```
def build_best_cnn(n_train_samples, batch_size=128, epochs=100):
    learning_rate = 0.003

    steps_per_epoch = n_train_samples // batch_size
    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=0.0,
        decay_steps=steps_per_epoch * epochs,
        warmup_target=learning_rate,
        warmup_steps=steps_per_epoch * 2,
    )

    inputs = tf.keras.layers.Input(shape=(28, 28, 1))
    x = inputs

    for filters in [64, 128, 128]:
        x = tf.keras.layers.Conv2D(filters, kernel_size=3, padding="same", activation="relu")(x)
        x = tf.keras.layers.BatchNormalization()(x)
        x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)

    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(64, activation="relu")(x)
    outputs = tf.keras.layers.Dense(4, activation="softmax")(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

cnn_model = build_best_cnn(n_train_samples=len(train_images))
cnn_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 64)	640
batch_normalization (BatchNormalization)	(None, 28, 28, 64)	256
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 14, 14, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 128)	512
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	147,584
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense_6 (Dense)	(None, 64)	73,792
dense_7 (Dense)	(None, 4)	260

Total params: 297.412 (1.13 MB)

```
train_ds_cnn, val_ds_cnn = make_datasets(
    x_tr_cnn, train_labels, sample_weights=sample_weights_full, batch_size=128
)

start = time.time()
cnn_model.fit(train_ds_cnn, validation_data=val_ds_cnn, epochs=100, callbacks=[early_stopping_cb], verbose=1)
t = time.time() - start

cnn_proba = cnn_model.predict(x_te_cnn)
cnn_pred = np.argmax(cnn_proba, axis=1)
evaluate_classifier("CNN", test_labels, cnn_pred, cnn_proba, t)
```


Epoch 1/100

686/686 — 18s 17ms/step — accuracy: 0.5860 — loss: 1.0303 — val accuracy: 0.5001 — val loss: 1.2183

x_tr_res = prep_for_cnn(train_images)

x_te_res = prep_for_cnn(test_images)

686/686 — 8s 11ms/step — accuracy: 0.7430 — loss: 0.6442 — val accuracy: 0.6582 — val loss: 0.8714

```
def build_best_resnet(n_train_samples, batch_size=128, epochs=100):
    learning_rate = 0.0008
    n_stages = 3
    units_per_stage = 2
    base_filters = 64
    activation = "relu"

    steps_per_epoch = n_train_samples // batch_size
    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=0.0,
        decay_steps=steps_per_epoch * epochs,
        warmup_target=learning_rate,
        warmup_steps=steps_per_epoch * 2,
    )

    inputs = tf.keras.layers.Input(shape=(28, 28, 1))
    x = tf.keras.layers.Conv2D(
        base_filters, kernel_size=3, strides=1, padding="same",
        kernel_initializer="he_normal", use_bias=False,
    )(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation(activation)(x)

    for stage in range(n_stages):
        filters = base_filters * (2 ** stage) # 64, 128, 256
        for unit in range(units_per_stage):
            strides = 2 if (unit == 0 and stage > 0) else 1
            x = ResidualUnit(filters, strides=strides, activation=activation)(x)

    x = tf.keras.layers.GlobalAvgPool2D()(x)
    x = tf.keras.layers.Dense(64, activation=activation)(x)
    outputs = tf.keras.layers.Dense(4, activation="softmax")(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

resnet_model = build_best_resnet(n_train_samples=len(train_images))

resnet_model.summary()

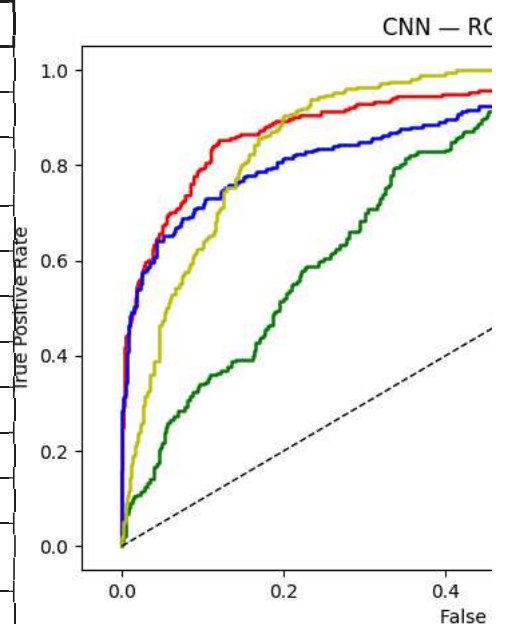
Weighted F1: 0.5548
Model: functional_3
Training Time: 196.7s

Layer (type)	CNN — Confusion Matrix	Param #
input_layer_3 (InputLayer)	(None, 28, 28, 1)	0
conv_normal_23 (Conv2D)	(None, 64, 28, 28, 4)	576
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 64)	256
activation_67 (Activation)	(None, 28, 28, 64)	0
residual_unit (ResidualUnit)	(None, 28, 28, 64)	74,240
residual_unit_1 (ResidualUnit)	(None, 28, 28, 64)	74,240
residual_unit_2 (ResidualUnit)	(None, 128, 14, 128)	230,912
residual_unit_3 (ResidualUnit)	(None, 128, 14, 128)	295,936
residual_unit_4 (ResidualUnit)	(None, 7, 7, 128)	920,576
residual_unit_5 (ResidualUnit)	(None, 21, 7, 128)	1,181,696
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense_8 (Dense)	(None, 64)	16,448
dense_9 (Dense)	(None, 4)	260

'acc': 0.595;
Total params: 2,795,140 (10.66 MB)

'f1': 0.5547871643497759.

train_ds_res, val_ds_res = make_datasets()



```

x_tr_res, train_labels, sample_weights=sample_weights_full, batch_size=128
)

start = time.time()
resnet_model.fit(train_ds_res, validation_data=val_ds_res, epochs=100, callbacks=[early_stopping_cb], verbose=1)
t = time.time() - start

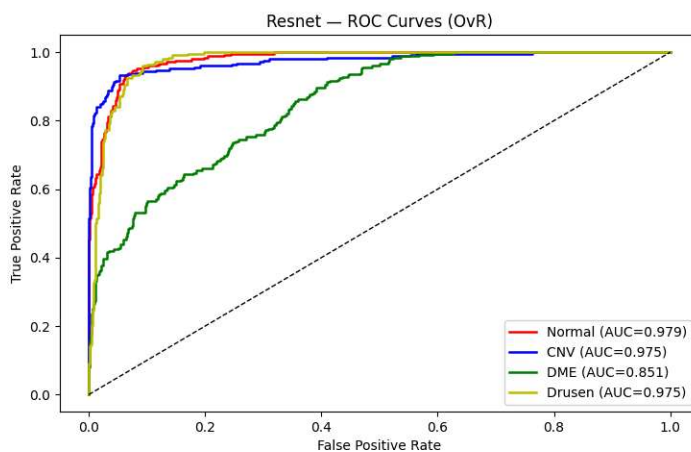
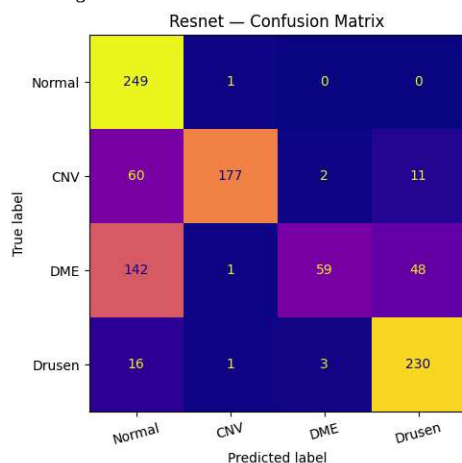
resnet_proba = resnet_model.predict(x_te_res)
resnet_pred = np.argmax(resnet_proba, axis=1)
evaluate_classifier("Resnet", test_labels, resnet_pred, resnet_proba, t)

```

```

Epoch 1/100
686/686 ————— 78s 91ms/step - accuracy: 0.6862 - loss: 0.8039 - val_accuracy: 0.8204 - val_loss: 0.5538
Epoch 2/100
686/686 ————— 53s 77ms/step - accuracy: 0.8223 - loss: 0.4919 - val_accuracy: 0.8667 - val_loss: 0.4600
Epoch 3/100
686/686 ————— 51s 75ms/step - accuracy: 0.8528 - loss: 0.4156 - val_accuracy: 0.7472 - val_loss: 0.6422
Epoch 4/100
686/686 ————— 52s 76ms/step - accuracy: 0.8678 - loss: 0.3674 - val_accuracy: 0.9080 - val_loss: 0.2677
Epoch 5/100
686/686 ————— 51s 75ms/step - accuracy: 0.8789 - loss: 0.3367 - val_accuracy: 0.7203 - val_loss: 0.7160
Epoch 6/100
686/686 ————— 51s 75ms/step - accuracy: 0.8898 - loss: 0.3013 - val_accuracy: 0.7539 - val_loss: 0.5131
Epoch 7/100
686/686 ————— 51s 75ms/step - accuracy: 0.8981 - loss: 0.2728 - val_accuracy: 0.8508 - val_loss: 0.4251
Epoch 8/100
686/686 ————— 51s 75ms/step - accuracy: 0.9086 - loss: 0.2396 - val_accuracy: 0.8484 - val_loss: 0.4148
Epoch 9/100
686/686 ————— 51s 75ms/step - accuracy: 0.9183 - loss: 0.2083 - val_accuracy: 0.8942 - val_loss: 0.3099
Epoch 10/100
686/686 ————— 52s 75ms/step - accuracy: 0.9295 - loss: 0.1758 - val_accuracy: 0.8335 - val_loss: 0.4802
Epoch 11/100
686/686 ————— 52s 75ms/step - accuracy: 0.9389 - loss: 0.1516 - val_accuracy: 0.9074 - val_loss: 0.2829
Epoch 12/100
686/686 ————— 52s 75ms/step - accuracy: 0.9490 - loss: 0.1269 - val_accuracy: 0.8551 - val_loss: 0.4190
Epoch 13/100
686/686 ————— 52s 75ms/step - accuracy: 0.9554 - loss: 0.1080 - val_accuracy: 0.8433 - val_loss: 0.4892
Epoch 14/100
686/686 ————— 52s 76ms/step - accuracy: 0.9618 - loss: 0.0948 - val_accuracy: 0.9178 - val_loss: 0.3062
32/32 ————— 3s 51ms/step
Resnet
Accuracy:          0.7150
Weighted Precision: 0.8086
Weighted F1:       0.6868
Training Time:     748.9s

```



```

{'name': 'Resnet',
 'acc': 0.715,
 'prec': 0.8085616655892108,
 'f1': 0.6867612359393038,
 'time': 748.919323682785,
 'auc': np.float64(0.9450226666666667)}

```